A. The reference-data design (ties to the INDICATOR_VALUE lecture)

1. All twelve status/type columns (WorkflowStatusID, StepStatusID, AccountStatusID…) are single-column FKs to one generalized ReferenceData table. The FK alone cannot stop a DOCUMENT_STATUS row from being stored as a workflow status. Where is the domain enforced — which triggers, on which events?

As of now, the domain is not fully enforced by a complete set of domain-validation triggers. While the FK column guarantees that WorkflowStatusID references some existing ReferenceData.ReferenceDataID, it doesn't guarantee that the domain name will specifically be 'WORKFLOW_STATUS'. We didn't realize but the seed scripts always seem to avoid the issue by resolving the IDs with the correct domain. The procedures resolve the controlled values by DomainName + Code so that way, the correct domains always use the correct domain. While I have a trigger for workflow terminations, it's not the necessary general-purpose domain-enforcement trigger for all the ReferenceData FKs. The general ReferenceData design is enforced primarily through the current controlled stored procedures and domain-qualified lookups but not database constraint for every ReferenceData foreign key. We have triggers to protect finalized history records and handle terminal workflow, but we need BEFORE INSERT/BEFORE UPDATE domain-validation triggers on all tables. Because the FK references only ReferenceDataID, a wrong-domain ID could theoretically be inserted through a direct table write. This is a large gap we need to resolve before our final submission.

2. Live test: using direct SQL (no stored procedure), insert a Workflow whose WorkflowStatusID points at a row from a different domain. What happens? If it succeeds, how would you close the hole?

When I tested it, I inserted a Workflow whose WorkflowStatusID points at a row from a different domain (where DOCUMENT_STATUS is 'active'), indicating an invalid workflow status (not what we wanted). The query succeeded, indicating that I do not have a domain validation trigger and my seed data always avoids this issue by selecting IDs with both DomainName and Code, with the current constraints only proving that the ID exists. To close this hole, I need BEFORE INSERT and BEFORE UPDATE triggers in place on every table that has generalized ReferenceData FKs. This is the most convenient fix, otherwise we would have to separate the table into separate domain-specific tables.

3. Why one generalized table instead of a lookup table per domain? What did you gain, what did you give up? When a status value is retired (IsActive = 'N'), what happens to historical rows that reference it in your reports?

By having the generalized ReferenceData, it simplified the schema and centralized control values but weakened type safety compared with having separate tables resulting in one lookup per domain. Essentially, by having the generalized table, we were able to maintain consistency and reduce schema duplication. It made adding new code within an existing domain easy: all we had to do was insert another ReferenceData row rather than changing the physical schema. It serves as a centralized location for controlled lookup values and UNIQUE(DomainName, Code) prevents duplicate domain codes within a domain. However, with this, we gave up stronger relational domain enforcement. Thus, we need extra enforcement through validation triggers and domain-specific procedures. With separate tables for domain lookup, it would have resulted in stronger FK domain integrity, clearer schema semantics, and simpler domain-specific constraints, even if it also requires many more tables and a more complex maintenance pattern with potential repetition. When a status value is retired or IsActive = 'N', it means the value can no longer be selected for business activity (the respective historical records are NOT deleted). Historical rows still contain the same ReferenceDataID so the FK remains valid and the historical relationship remains intact. Reports should join by ID without filtering on isActive so historical records are preserved. IsActive controls whether the value may be chosen for new transactions.

B. Workflow engine semantics

4. In a parallel step, what makes the workflow advance — all participants, or all *required* participants (RequiredFlag)? Show the procedure logic that decides.

A parallel step advances when all required participants (RequiredFlag = TRUE) have completed their assignments, not necessarily every participant. When a participant completes an action, the corresponding procedure (sp_complete_review_acknowledgement(), sp_record_approval_decision(), or sp_record_signature()) marks their assignment as COMPLETED and calls sp_advance_workflow(). Within that procedure, fn_step_required_work_complete() checks whether any required assignments remain incomplete. It returns TRUE when no required assignments remain incomplete and at least one required assignment exists. This result is used in the following UPDATE statement to complete qualifying active steps:

```
UPDATE WorkflowStep

SET StepStatusID = v_step_completed_id,
```

```
    CompletionDateTime = CURRENT_TIMESTAMP
WHERE WorkflowID = p_workflow_id
  AND StepStatusID = v_step_active_id
  AND fn_step_required_work_complete(WorkflowStepID) = TRUE;
```

Once all steps in the current sequence group are completed, sp_advance_workflow() either completes the workflow or activates the next eligible sequence group, provided its predecessor requirements have been satisfied. Optional participants therefore do not prevent the workflow from advancing.

5. Two participants act on the same step at the same moment. What prevents a race or a double completion? Which isolation level / locking does the action procedure rely on?

The action procedures use InnoDB transactions and explicit SELECT ... FOR UPDATE row-level locking to prevent conflicting actions. At the beginning of each procedure, a locking query retrieves the relevant WorkflowParticipant, WorkflowStep, Workflow, and reference-data records. This ensures that concurrent transactions cannot independently process the same assignment.

For example, if two requests attempt to complete the same participant assignment at the same time, the second transaction must wait for the first to finish. Once the first transaction commits, the second request proceeds and checks the assignment's current status. Since the assignment is no longer ACTIVE, the duplicate action is rejected.

The procedures do not explicitly set an isolation level, so they rely on InnoDB's default REPEATABLE READ isolation level. The main protection comes from the explicit row locks and status checks rather than the isolation level alone. These protections are implemented in sp_complete_review_acknowledgement(), sp_record_approval_decision(), and sp_record_signature().

6. Workflow expiration: there is no MySQL EVENT — sp_expire_workflows must be called. Who calls it in production, and until it runs, can a participant still sign a workflow whose ExpirationDateTime has already passed? Do the action procedures check expiry themselves?

There is no MySQL EVENT configured to execute sp_expire_workflows() automatically, so in production an external scheduler, such as an application-level scheduled job or cron job, would need to call the procedure periodically. Until it runs and changes the workflow status to EXPIRED, participants can still act on a workflow even if its

ExpirationDateTime has already passed. This is because the three action procedures check whether the workflow, step, and participant assignment are ACTIVE, but do not independently compare the expiration timestamp against CURRENT_TIMESTAMP. To enforce expiration immediately, those procedures would also need an expiration-time check. The database does have an index, idx_Workflow_Status_Expiration, on (WorkflowStatusID, ExpirationDateTime), which helps make the scheduled expiration scan more efficient.

7. The terminal-cascade trigger cancels remaining steps when a workflow is rejected/expired/cancelled. Why a trigger rather than procedure logic — and what happens if the cascade itself fails mid-transaction?

The terminal-cascade trigger, trg_Workflow_TerminalCascade, automatically cancels unfinished child records whenever a workflow transitions to CANCELLED, EXPIRED, or REJECTED. It fires after an update to Workflow and checks whether the status actually changed to one of those terminal states. It then marks any remaining PENDING or ACTIVE workflow steps and participant assignments as CANCELLED, while leaving completed records untouched to preserve history.

We implemented this as a trigger so that the cleanup happens regardless of which code path terminates a workflow. Our termination procedures (sp_cancel_workflow(), sp_record_approval_decision(), and sp_expire_workflows()) also perform child-record cancellations themselves. This creates some overlap, but the trigger provides additional protection if a workflow is terminated through another code path that does not explicitly handle its child records.

If the cascade fails midway, the original workflow update also fails because MySQL triggers execute within the transaction that caused them to fire. The exception handler then executes ROLLBACK and RESIGNAL, ensuring that the workflow status change and child-record cancellations either succeed together or roll back together. Test 10 demonstrates this by directly updating a workflow to a terminal status and verifying that its unfinished steps and participant assignments are also cancelled.

C. Immutability, audit, and the meaning of a "signature"

8. Signatures and decisions are immutable via triggers, and AuditLog is append-only. What stops a DBA with full privileges from rewriting history anyway — and is that in scope for your threat model?

A DBA with full privileges is outside the current threat model. The current triggers protect against ordinary and privileged SQL paths that execute the table’s triggers. Essentially, our immutability and append-only controls are designed to prevent

unauthorized modification through normal database operations and application paths. They are not intended to defend against malicious DBA paths with unrestricted root-level control, because that administrator can alter the very rules enforcing immutability. Indicating it is outside the project's threat model. For stronger protection, protecting against privileged administrators would require controls outside the transactional schema itself– a separation of duties, restricted admin roles, tamper-evident or external audit storage, signed audit records, database activity monitoring, and organizational access controls. Our current design provides database-level integrity, not tamper-proofing against the database owner.

9. What does a Signature row actually prove? There is no document hash — if the underlying document content changed after signing, how would anyone detect it? What field would you add to make a signature tamper-evident?

The Signature row essentially proves who signed the Document, when they signed, which Workflow assignment they signed under, and which DocumentVersionID the signature is associated with. It doesn't necessarily cryptographically prove that there has been no modifications made to the underlying bytes since signing. The current design follows logical binding between the Signature and the DocumentVersion, but does not provide cryptographic binding (preserving Signature integrity). Thus, if the underlying document content changed after signing, the schema would not be able to detect it just from the Signature row. To make the signature tamper-evident, we would have to add a field such as DocumentHash or just ContentHash. Essentially, we would use a hash like SHA-256 to compute the content hash and store it for comparison to make sure the signature content has not been tampered with (stored with respect to that specific DocumentVersion). So to verify integrity, it recomputes SHA-256 over the Document content, compares it to the ContentHash, and if the hashes differ, it indicates that the content has changed. As the hash is a property of the DocumentVersion itself, we should add it as a property of DocumentVersion once it is locked, not Signature. This process would cryptographically bind the signature to the exact document contents instead of just the version identifier. The current LOCKED trigger doesn't resolve this issue fully– it's not enough. While it may prevent normal updates to the DocumentVersion, if the content lives elsewhere, a hash is still necessary to protect from malicious content changes.

10. AuditLog stores PriorValueContext / NewValueContext as text. Why not structure (JSON or per-column) — how would you answer "show me every change to this document's status in March" efficiently?

PriorValueContext and NewValueContext are easy to populate from many different business events because AuditLog is generic. A workflow cancellation, approval rejection, document update, signature event, and version lock can all write before/after content descriptively without needing a different audit entry for every entity. It provides flexibility, allowing for easy human-readable evidence and a simple append-only history. However, it does sacrifice structure; so a question like ‘Show me every change to this document’s status in March’ is not currently supported by the database design. With the current schema, we would have to write a query that narrows the audit history by record type, record ID, and date range. But identifying only the document’s status changes is more difficult as it becomes buried in text and textual conventions can change. Thus, a JSON with index generated fields would be better so we can query explicitly, making it more reliable than parsing free text. Per-column auditing could also work as we could model a child table ‘AuditChange’ where one business update could produce what exact changes have occurred within the event time indicated by the ‘AuditLog’, rendering the March query trivial, but adding another table to the schema and additional complexity.

D. Transactions and testing

11. Walk through TX-02 (Complete Workflow Action): which writes must succeed or fail together, and show the exception test that proves the rollback.

TX-02 represents the transaction involved when a participant completes an assigned workflow action. It is implemented through sp_complete_review_acknowledgement(), sp_record_approval_decision(), and sp_record_signature(). Taking sp_record_signature() as an example, the procedure first locks and validates the participant's assignment, checking the assigned user, action type, active status, document version, authorization, and predecessor requirements. It then inserts the Signature record, marks the participant's assignment as COMPLETED, inserts the corresponding AuditLog record, and calls sp_advance_workflow() to determine whether the step or workflow can advance. These operations must succeed or fail together so that a signature cannot exist without its audit entry, a participant cannot be marked completed without a successfully recorded action, and the workflow cannot advance based on an action that failed. Each action procedure has an exception handler that executes ROLLBACK and RESIGNAL if an operation fails, including one inside sp_advance_workflow(). For testing, TEST 4 (sp_test_04_mid_transaction_rollback) demonstrates this rollback mechanism on TX-01, the workflow initiation transaction, rather than TX-02. The test installs a temporary trigger that deliberately raises an exception during the second participant insertion, after the workflow, first step, and first participant have already been inserted. It then verifies that the Workflow, WorkflowStep, and WorkflowParticipant record counts return to their original values. This demonstrates

the general rollback behavior, but the action procedures currently do not have a dedicated test that deliberately forces a failure midway through TX-02.

12. Your exception tests — pick one and explain what production failure it simulates. Which business rule has *no* negative test, and why?

One example is TEST 1 (sp__test_unauthorized_approval), which tests an authorization failure. In this scenario, Marcus Reed attempts to submit an approval using Daniel Cho's active participant assignment, even though Marcus is not the assigned user. This simulates a possible production issue where the application passes the wrong user ID or incorrectly associates an authenticated session with another participant's assignment. The procedure checks that the person attempting the action matches the user assigned to that participant record, so the mismatch causes the action to be rejected. The test verifies that no ApprovalDecision record is created and that the participant assignment and workflow remain active. One important business rule that lacks a negative test is expiration enforcement during participant actions. TEST 9 verifies that sp_expire_workflows() can transition past-due workflows to EXPIRED, but there is no test checking whether someone attempting to sign or approve an expired-but-unprocessed workflow is rejected. In fact, that action would currently be allowed because the action procedures do not independently check ExpirationDateTime. We also lack direct negative tests for the immutability triggers protecting Signature and ApprovalDecision records; TEST 6 and TEST 7 cover audit-log and locked-document-version protections, respectively.

13. The build resolves a Git tag to a commit SHA. Why does that matter for grading/reproducibility — what broke in your early builds (delimiter and trigger issues in the Oct 24 commits) and what did you change?

The build process resolves the requested Git tag to a specific commit SHA and records that SHA in the build log. This matters because a Git tag is a label that can technically be moved to another commit, whereas a commit SHA identifies an exact source revision. Recording the SHA allows someone grading the project to identify precisely which version of the code was built and reproduce that build later. During our October 24 builds, we encountered issues involving the terminal-cascade trigger and SQL delimiter handling. The trigger implementation needed revisions, and there was also a delimiter mismatch: the trigger used $$ while the rest of the file and build process used //, which caused problems when parsing and executing the SQL scripts. We corrected the trigger implementation and standardized its definition and DROP statement to use the // delimiter. These fixes were carried through the subsequent test releases, including v0.1.7-test and v0.1.8-test, before the final v1.0.0 release. We also made delimiter declarations explicit in each SQL file so the build process can correctly parse each

script. Resolving the tag to a commit SHA ensures that the grader can reproduce the exact final version rather than relying on a potentially movable tag name.

E. Individual accountability (ask each member about the other's subsystem)

14. Ismael: explain the terminal-cascade trigger and the version-locking design.

The terminal-cascade trigger, trg_Workflow_TerminalCascade, automatically cancels unfinished child records whenever a workflow transitions to CANCELLED, EXPIRED, or REJECTED. It runs after an update to the Workflow table and checks that the workflow's status actually changed to one of those terminal states. It then updates any PENDING or ACTIVE WorkflowStep and WorkflowParticipant records associated with that workflow, marking them as CANCELLED. Completed records remain unchanged to preserve historical information, and the trigger uses COALESCE to avoid overwriting existing completion timestamps. The reason we implemented this as a trigger is to centralize the cleanup logic and ensure that it applies regardless of which code path terminates a workflow. All three termination procedures (sp_cancel_workflow(), sp_record_approval_decision(), and sp_expire_workflows()) already perform their own child-record cancellations, so the trigger overlaps with existing procedure logic, but its status filters make the repeated updates harmless. It also acts as a safeguard against future code paths that might terminate a workflow without explicitly cleaning up its children. Because the trigger executes inside the transaction that caused it to fire, a failure in the cascade also fails the workflow update, and the procedure's exception handler rolls back the transaction and propagates the error. Test 10 demonstrates this independently by directly updating a workflow to a terminal status, bypassing sp_cancel_workflow(), and verifying that its unfinished steps and participant assignments are also cancelled.

The version-locking design ensures that approvals and signatures are associated with a specific document version rather than a document whose contents might change during the approval process. When creating a new version, sp_create_document_version() locks the parent Document row using SELECT ... FOR UPDATE. This prevents concurrent requests from creating conflicting versions for the same document. It also checks whether a CURRENT version already exists and rejects creation if one does. This one-CURRENT-version rule is enforced through procedure logic rather than a unique database constraint, meaning direct inserts that bypass the procedure could violate it. When a workflow is initiated, sp_initiate_workflow() verifies that the requested document version is CURRENT, changes its status to LOCKED, and stores its DocumentVersionID on the workflow. These operations happen within the same transaction. Once locked, database triggers prevent the document version from being updated or deleted, as demonstrated by Test 7. When a participant signs,

sp_record_signature() checks that the version being signed matches the exact DocumentVersionID attached to the workflow. A foreign key also ensures that the referenced version exists. Additional triggers protect Signature and ApprovalDecision records from modification or deletion. One important design decision is that a locked version does not return to CURRENT if its workflow is cancelled, rejected, or expires. Instead, the document owner must create a new version. This preserves the original version as a historical record and ensures that the document being approved is not silently changed or reused.

Sanika: explain the validation/smoke-test strategy and the entitlement matrix.

This system has three validation layers (Object, Data, smoke testing) and the entitlement matrix answers who is allowed to do what, to which records, under which lifecycle conditions, and where the authorization is enforced.

The first validation script was 010_object_validation.sql. The purpose of this script is to verify that the expected database objects exist by querying the information_schema. The database objects include base tables, indexes, views, routines, triggers, and database roles. It is deliberately fail-closed, so if something is missing, it will raise an ER45000 so the build fails quietly instead of producing a partially installed database. The object validation checks things like making sure the required tables exist, required views exist, required functions exist, required procedures exist, required triggers exist, required database roles exist, and important PK/UNIQUE/FK/index definitions exist. So, for example, if the WorkflowParticipant table exists but the composite FK connecting (UserID, RoleID) to UserRole is missing, the schema would not implement our design correctly, thus object validation is here to detect that sort of installation defect. The script has two main useful aspects, emitting text like ‘Workflow -> PRESENT’ and a hard check (SIGNAL SQLSTATE '45000'). So, the result sets make validation understandable to a human while SIGNAL checks make validation enforceable by automated build.

The second validation script was 020_data_validation.sql. The purpose of this script was checking that given that the structure exists, does the data obey the model, essentially moving from metadata to content. The script explicitly describes its scope as verifying seeded content, reference-data completeness, FK/uniqueness consistency, role assignment correctness, and workflow-state invariants. The data validation checks that required core data exists– verifying that seeded tables aren’t unexpectedly empty. It also checks that required reference codes are present and active (not resolving the FK domain hole issue from earlier but making sure the values needed by implementation are available), required business roles exist, and that seed scenarios actually cover lifecycle states. The validation also checks that relationship consistency; for example, checking that every WorkflowParticipant’s (UserID, RoleID) exists in UserRole. This

validation also checks cross-table business consistency to validate relationships that aren't sufficiently expressed by 'row exists', such as checking consistency with participant action matching the corresponding WorkflowStepType. Validation also checks Workflow state invariants, making sure the seeded database isn't inherently contradictory.

The last layer was the smoke tests (030_smoke_tests.sql). This script tests to see if the built database is actually usable– this includes representative read queries over operational values as well as a few guarded assertions. Essentially, we are checking to make sure that the view actually returns useful/correct business information. The first smoke test was for outstanding actions, checking specifically that the operational reporting works to retrieve the incomplete work for Document, Workflow, Participant, and more. The second smoke test was for the participant-specific worklist, to retrieve workflow work associated with one specific participant, with the third smoke test providing the document history querying several layers and outputting review, approval, and signature states across the workflow. The fourth smoke test is for the completed workflow detail– describing the steps, participants, roles, actions, decisions, and signature result, with the fifth smoke test asserting version-history completeness. This makes sure every stored document version would remain visible through the historical read model. Smoke tests 6 and 7 are for approval reporting and signature consistency respectively, checking that they obey the exact-version binding rule. The last two smoke tests assert audit-history readability and performance/access-path behavior to correctly retrieve active assignments for a user.

The entitlement matrix separates business eligibility from assignment-specific authority. It answers what operation the role can attempt, on what resource, and under what row/state restrictions. Authorization is layered; we must check that the user has the correct business role, the exact work that is assigned to the user, the workflow/step/assignment is in the right state, and that all preceding steps have been completed before concluding that the user action is authorized. The roles in the entitlement matrix include the Document Owner, Reviewer, Approver, Signer, Workflow Manager, System Administrator, and Auditor. For the Document Owner, the matrix describes conditions regarding modifying ONLY the document they own, as well as only being able to modify the workflow associated with the document they own. Similarly, for the Reviewer and Approver, they can only perform their respective roles with the specific Workflow/step/assignment they are assigned to, with the right state (Active), and with preceding steps complete. Additionally for Signer, the signature must be bound to the exact DocumentVersion. The WorkflowManager and Auditor have specific access and monitoring privileges within their authorized scope with an emphasis on Auditor having least-privilege role. Lastly, the matrix describes conditions for the System

Administrator for administrative authority, but NOT ordinary permission to rewrite completed approvals, signatures, or audit evidence.

15. Either: pick one FR from the traceability matrix and trace it live —
    FR → schema object → enforcement → test case → demo evidence.

Tracing FR-11 (Record a Signature Event) from the traceability matrix:

1. Starting from the Functional Requirements Document:

FR-11 — High Priority:

The system shall record an electronic signature for an assigned signer, including the signer, exact document version signed, signing date/time, and signature status or method.

Related entities: WorkflowParticipant, Signature, DocumentVersion.

Essentially, the system must preserve evidence of who performed the signing action, which exact document version was signed, when the action occurred, and the resulting signature status or method. The signature must belong to the participant specifically assigned to the signing step and must reference the same document version bound to the workflow.

Acceptance Criteria:

The FRD acceptance criteria for FR-11 require the system to demonstrate that a valid signature can be recorded and later retrieved together with:

- Assigned signer
- Exact document version signed
- Signing date/time.
- Signature status or method.

The evidence should include the transaction used to record the signature and a joined query showing the signer and exact DocumentVersion associated with that signature.

For negative validation, the implementation should also demonstrate that a signature cannot be recorded against an incorrect document version or by an unauthorized participant.

Acceptance evidence used in this implementation:

- A correct signature attempt succeeds and creates exactly one Signature row
- The stored DocumentVersionID matches the version bound to the workflow
- The signature can be joined back to the assigned signer through WorkflowParticipant
- A wrong-version signature attempt is rejected
- A duplicate signature for the same assignment is rejected
- The successful signature cannot later be updated or deleted through ordinary operations

2. Schema Object

Location: database/01_schema/030_relationship_tables.sql

The Signature table stores:

- WorkflowParticipantID — identifies the specific workflow assignment that performed the signature.
- DocumentVersionID — identifies the exact document version signed.
- SigningDateTime — records when signing occurred.
- SignatureStatusID — stores the controlled signature status.
- SignatureMethod — stores the signing method.
- SignatureReference — provides an identifying reference for the signature event.

The signer is not duplicated directly inside the Signature table. Instead, signer identity is obtained through:

Signature.WorkflowParticipantID → WorkflowParticipant.UserID

This ties the signature to the exact workflow assignment rather than merely to a user.

Important integrity constraints include:

- uq_Signature_WorkflowParticipant — prevents more than one signature from being recorded for the same participant assignment.
- chk_Signature_Method_NotBlank — prevents a blank signature method.
- Foreign keys ensure that the referenced participant assignment and document version exist.

The signature is associated with both an **assignment** and an exact **DocumentVersion**. This preserves the context of what the user was authorized to sign and which version was actually signed.

3. Enforcement

Location: database/02_programmability/030_procedures.sql
Procedure: sp_record_signature

Before a signature is inserted, the procedure validates that:

- The current user is the user assigned to the WorkflowParticipant.
- The assigned user holds the required Signer business role.
- The workflow is ACTIVE.
- The signature step is ACTIVE.
- The participant assignment is ACTIVE.
- Required predecessor work has been completed.
- The signature action has not already been completed.
- The submitted DocumentVersionID matches the exact version bound to the workflow.
- The signature method satisfies required validation.

If validation succeeds, the procedure:

- Inserts the Signature row.
- Marks the participant assignment complete.
- Writes the corresponding audit event.
- Invokes workflow progression logic so that the workflow can advance or complete when appropriate.

Additional historical protection is implemented in database/02_programmability/040_triggers.sql:

- trg_Signature_PreventUpdate
- trg_Signature_PreventDelete

These triggers make a recorded signature **immutable** through ordinary database operations.

Enforcement distinction:

- The stored procedure prevents an invalid signature from being created.
- Constraints prevent structurally invalid or duplicate signature records.

- Triggers prevent valid historical signature records from later being changed or deleted

4. Test Case

Main test file: database/06_demo_tests/080_resiliency_demo.sql

FR-11 is exercised through positive, negative, and duplicate-signature cases.

TC-FR11-N01 — Wrong Document Version

Sofia Alvarez attempts to sign using a DocumentVersionID that belongs to a different document/workflow context.

Expected result:

- The operation is rejected.
- No Signature row is created.
- The participant assignment remains unchanged.
- Workflow state remains unchanged.

TC-FR11-P01 — Correct Signature

Sofia signs using the exact DocumentVersionID associated with the workflow.

Expected result:

- The operation succeeds.
- A Signature row is created.
- The signature references Sofia's assigned WorkflowParticipant.
- The stored DocumentVersionID equals the workflow's DocumentVersionID.
- Signing date/time, status, and method are preserved.
- The participant assignment completes and workflow progression occurs.

TC-DUP-02 — Duplicate Signature

Sofia attempts to sign the same participant assignment again.

Expected result:

- The second attempt is rejected.
- No second Signature row is created.

- uq_Signature_WorkflowParticipant and procedure-level state validation prevent duplicate signing.

Supporting validation evidence

- Smoke Test 7 in 030_smoke_tests.sql verifies that every recorded signature references the same DocumentVersionID as its workflow.
- Data validation checks the same signature/version relationship.
- Data validation verifies that each participant assignment has at most one signature.
- Signature immutability tests verify that completed signature rows cannot be updated or deleted.

5. Demo Evidence

The demo uses four participants:

- Olivia Brooks — Document Owner
- Marcus Reed — Reviewer
- Daniel Cho — Approver
- Sofia Alvarez — Signer

The workflow proceeds through review, approval, and signing. CHECKS 11–13 demonstrate the three FR-11 cases: wrong version rejected, correct version accepted, and duplicate signature rejected.

To verify the stored signature, query Signature alongside WorkflowParticipant and Workflow and compare s.DocumentVersionID with w.DocumentVersionID. They should match.

Overall traceability: FR-11 defines the required signing behavior. The Signature, WorkflowParticipant, and DocumentVersion schema objects store the required facts. sp_record_signature enforces authorization, lifecycle, predecessor, and exact-version rules. Constraints prevent malformed or duplicate data, while triggers protect completed signature history. Positive, negative, duplicate, smoke, and validation tests provide evidence that the requirement is implemented correctly.